Test Plan

Areas ranked by impact

1. Ergonomic scoring & risk calculations (RAMP, RULA, REBA, MEC)

Backend: `ramp/`. *Frontend:* `features/RAMP`, `features/RULA` *scoring utils.*

This is the product. A worker's exposure to injury risk is reported as a colour band (green/yellow/red) computed from published ergonomic tables. If a score is wrong, the app tells a safety manager an unsafe posture is fine, or flags a safe one as high-risk — the core value proposition fails silently, with no error to alert anyone. Ranked highest because it's both the most complex logic in the codebase (table lookups, threshold bands, per-question exceptions) and the one place where "wrong" doesn't crash, it just misinforms.

2. Session data pipeline (ingestion, merging, signal processing)

Backend: `user_sessions/` *— merge, resample, signal algorithms, MEC neck/hand pairing.*

Everything in area 1 is computed from this data. If sensor merging, resampling, or timestamp pairing is wrong, the scoring logic runs correctly on corrupted input and produces a confident, wrong answer — worse than a visible failure, because nothing looks broken. Ranked second because a bug here silently poisons every score downstream of it, not just one feature.

3. Multi-tenant permissions & org scoping

Backend: `user_sessions/test_session_permission.py`, `test_session_sync_worker_org.py`, `organizations/`, `users/test_scoping.py`.

A session or worker action scoped to the wrong organization means one company can see or modify another company's worker data — a trust and compliance failure, not just a UX bug. Ranked third because the blast radius is cross-customer, even though it's less complex than areas 1–2.

4. Reports & exports (DOCX/Excel reports, PDF comparison exports)

Backend: `user_sessions/test_excel_export.py`, `test_docx_helpers.py`. *Frontend:* `MultiResultsContainer` *PDF export.*

These are the contractual deliverable a customer actually walks away with. A wrong number in an exported report is the same failure as area 1, except it's now on paper/PDF in the customer's hands with no way to silently correct it later. Ranked below areas 1–3 because the underlying scores are already covered there — this layer mostly needs to verify the export doesn't corrupt or mis-render already-correct data.

5. Authentication & account management

Backend: `firebase_auth/`, `organizations/test_create_profile.py`, `test_update_user_email.py`, `users/test_permission_levels.py`.

Broken auth locks users out or lets an account action fail ungracefully. Ranked lower than areas 1–4 because failures here are loud — users notice immediately and report it — rather than silent data corruption.

6. Route access & i18n

Frontend: `AppRoutes.test.tsx`, `i18n.test.ts`.

Confirms signed-out routes (invitations, privacy pages) render, and translations resolve per-language without blanking the UI. Ranked lowest of the tested areas: a broken translation or route is visible and annoying, but doesn't produce wrong data or a security gap.

Deliberately not testing

Backend — apps with zero test files:

- `tno/` — the calculation engine for the TNO assessment type. Not just uncovered: `tno/tests.py` contains a fully written test suite (calculation logic and 5 API endpoint cases) that has been entirely commented out rather than deleted or maintained.
- `devices/` — device registration/pairing, permissions, and throttling logic. No tests despite having its own `permissions.py` and `throttles.py`.
- `scheduler/` — background job scheduling. No tests.
- `utils/` — shared email/email-i18n helpers used across the app. No tests.

Frontend — features with zero test files:

- `REBA/` **and** `MEC/` — these are ergonomic scoring features, the same category as RAMP and RULA (area 1, ranked highest-impact) — but unlike RAMP/RULA, they have no tests at all. This is a real gap inside the highest-priority area, not a deliberate exclusion, and should be closed before lower-priority work.
- `auth/` **and** `sessions/` — no frontend tests, even though the backend equivalents (`firebase_auth/`, `user_sessions/`) are the most heavily tested backend apps. The logic is covered server-side, but nothing verifies the frontend auth/session UI behaves correctly against it.
- **Load/performance testing** — out of scope; nothing currently indicates this is a risk area.

Constraint status

- **Placeholder tests deleted** — done. `client-panel/App.test.tsx` and `wergonic-admin/App.test.tsx` both asserted only `expect(true).toBe(true)`; removed on `chore/remove-placeholder-app-tests`.
- **Tests block the pipeline** — verified, not assumed. Backend `build` job has `needs: test`; frontend `docker` job has `needs: [set-envs, build]` with `build` running `vitest run`. Confirmed with a deliberately broken assertion pushed to a throwaway branch: `Test the backend` → `failure`, `Build and Publish Docker image to DO` → `skipped`.